Contents

Eng Labs — Multi-Tenant EdTech Platform **1**
Problem . . . 1
1. The system at a glance . . . 2
2. Monorepo architecture & tooling . . . 2
3. Runtime topology . . . 4
4. The stack, by layer . . . 7
5. Multi-tenancy — the core architectural decision . . . 7
6. Request lifecycle . . . 8
6.5 Domain modelling — Project Mentor as the worked example . . . 8
7. Skills demonstrated . . . 10
8. Known gaps in *this document's* accuracy . . . 13
9. Which roles these skills open . . . 13
10. How these roles & skills evolve with AI — the honest read . . . 13
Outcome / impact . . . 15
Skills demonstrated (summary for the matrix) . . . 15
Artifacts . . . 15

Eng Labs — Multi-Tenant EdTech Platform

- **Date:** 2026-07 (ongoing)
- **Context:** Primary work project — a multi-tenant SaaS platform for engineering education & placements
- **My role:** Full-stack / platform engineer — built and own large parts of the backend, frontend, AI services, data model, and the CI/security tooling
- **Team:** ~8 contributors, ~2,530 commits since Nov 2025. I am the largest single contributor (~29% of commits).

This is my flagship evidence entry and the **source of truth for the system's architecture**. It is deliberately detailed because the codebase is large and touches many disciplines. Use it as the anchor when presenting myself; the Skills Matrix points here for most rows.

Companion notes: multi-tenancy — the *concepts* (five isolation models, enforcement layers, leak paths, interview answers) and the *lessons* from this codebase (mistakes, tradeoffs, roadmap). user-experience — the 13-check UX review list and the role-vs-job surface design case study.

All figures below are measured from `main`. See §8 for why that matters.

Problem

Build a production platform that serves multiple institutions (colleges/companies) from one codebase, covering five distinct products: an LMS/courses engine, a digital exam & evaluation system, a capstone "project mentor," a placements suite (mock coding assessments + AI mock interviews), and the platform core (auth, org management, admin). Every institution's data must stay isolated, every product must be independently toggleable per customer, and the whole thing must be secure and auditable enough to pass ISO 27001 change-management.

---

1. The system at a glance

Measured on `main`, 2026-08-09. Keep this table current — it is the fastest interview reference in the whole knowledge base.

Dimension	Figure
Apps	7
Shared packages	8
Cloud functions	8
TypeScript / TSX (first-party)	**477,278 lines**
Python (interview service)	13,398 lines
Prisma models	**199** (6,444-line schema)
API route files / endpoints	**198 / 586**
`responseWrapper` call sites	792
API test files	176
Coverage gate	90% branches + statements
GitHub Actions workflows	8
Contributors / commits	~8 / ~2,530

Per-workspace size:

Workspace	Lines	What it is
`apps/api`	240,638	Express + TypeScript backend (includes tests)
`apps/dashboard`	150,714	Main Next.js 16 frontend
`packages/*`	58,757	8 shared packages
`apps/mock-oa`	19,007	Standalone coding-assessment app
`apps/interview-service`	13,398 (py)	FastAPI real-time AI interview service
`apps/platform-admin`	8,162	Super-admin panel

---

2. Monorepo architecture & tooling

The structural layer of the system — and the part most worth being able to explain from first principles, because it transfers to every future project.

The key insight: these are four independent layers, not one thing.

Layer	What it decides	Realised as
Monorepo	Many projects, one Git repo	the folder layout — nothing more
Workspace	Which folders are projects; link them so they can `import` each other	`pnpm-workspace.yaml` (JS) + `pyproject.toml`/`uv.lock` (Python)

Layer	What it decides	Realised as
Task runner	Which scripts run, in what order, and whether to skip them	`turbo.json`
Scripts	The actual work	`tsc`, `next build`, `prisma generate`, `jest`, `uvicorn`

Each is replaceable. Swap pnpm for npm and Turbo still works; delete Turbo and pnpm still links; delete both and it is still a monorepo.

What each root file does

```
package.json          → repo-wide tooling + entry-point scripts (turbo dev/build/lint)
pnpm-workspace.yaml   → which folders count as projects
pnpm-lock.yaml        → exactly what got installed, for ALL of them (one per workspace)
turbo.json            → how to run their scripts: order, caching, env inputs
pyproject.toml        → the Python manifest (a second, separate workspace)
```

How pnpm actually shares code

`"@repo/database": "workspace:*"` does **not** download from npm — pnpm symlinks the local folder:

```
apps/api/node_modules/@repo/database -> ../../../../packages/database
```

pnpm stores each third-party package **once** in a content-addressable store and symlinks it in, rather than copying it per project. It also **forbids phantom dependencies**: if a package isn't in your `package.json`, you can't import it. That strictness is why pnpm is the standard choice for monorepos.

How Turborepo actually works

Turbo does **not** understand TypeScript. It `cd`s into a folder and runs a command in a shell — it "shells out." Its entire contribution is three things:

1. **Order** — derived from `package.json` dependencies plus `"dependsOn": ["^build"]`. Never written by hand.
2. **Parallelism** — a pipeline, not waves: each task starts the moment *its own* dependencies finish.
3. **Caching** — hashes every input file *and* the declared env vars. Critically, the hash **includes each dependency's hash**, so a change deep in the graph correctly invalidates everything downstream.

`turbo build --dry=json` prints the whole plan without executing — the fastest way to see the real graph.

The workspace dependency graph

Acyclic and cleanly layered — apps on top, `@repo/database` at the foundation. Nothing points sideways or upward. This is the structurally strongest part of the repo.

Package	Depends on	Purpose
`@repo/database`	—	Prisma client, 199 models, CockroachDB
`@repo/config`	—	API URLs, service URLs, feature keys

Package	Depends on	Purpose
`@repo/analytics`	—	Zod event schemas for learning analytics
`@repo/telemetry`	—	OpenTelemetry traces/metrics/logs
`@repo/ui`	database	Shared component library
`@repo/helpers`	telemetry, database	Auth guards, storage, email, errors
`@repo/placements`	config, database, helpers	Placements domain (server + client + common)
`@repo/mass-customization`	config, database, ui	Courses/LMS domain

What is deliberately *outside* the workspace

`cloud-functions/*`, `apps/resume`, and `apps/interview-service` are in the repo but not in the pnpm workspace — they have their own lockfiles (or none). **This is a real tradeoff, not an oversight:** they deploy independently, so isolation is a feature. The cost is measurable — `express` has already drifted (`^4.18.2` in four functions, `^4.22.1` in a fifth), and `ts-jobspy` is at `^1.4.0` in `job-scouting-ingest` while `@repo/placements` is on `^2.0.3`.

Consequence to remember: `pnpm dev` starts the JS apps and package watchers. It does **not** start the Python interview service — that has no `package.json`, so Turbo cannot see it. Run it separately with `python main.py`.

---

3. Runtime topology

Service	Port	Server model
`apps/api`	8080	Express — HTTP server is **in-process** (`app.listen`)
`apps/dashboard`	3000	Next.js built-in server
`apps/platform-admin`	3001	Next.js
`apps/mock-oa`	3002	Next.js
`apps/interview-service`	8090	FastAPI — needs **uvicorn**, a separate ASGI server

Worth being able to explain: in Node the HTTP server is part of the runtime, so `app.listen()` *is* the server starting. In Python, FastAPI only *defines* routes — uvicorn (ASGI) or gunicorn (WSGI) serves them. That is why the Dockerfile's `CMD` is `uvicorn main:app`, and why the Node services have no equivalent line.

Deployment: Cloud Build → Artifact Registry → Cloud Run, per service (`infra/cloudbuild*.yaml`). No Terraform or Kubernetes — infrastructure is declarative YAML per service, not IaC.

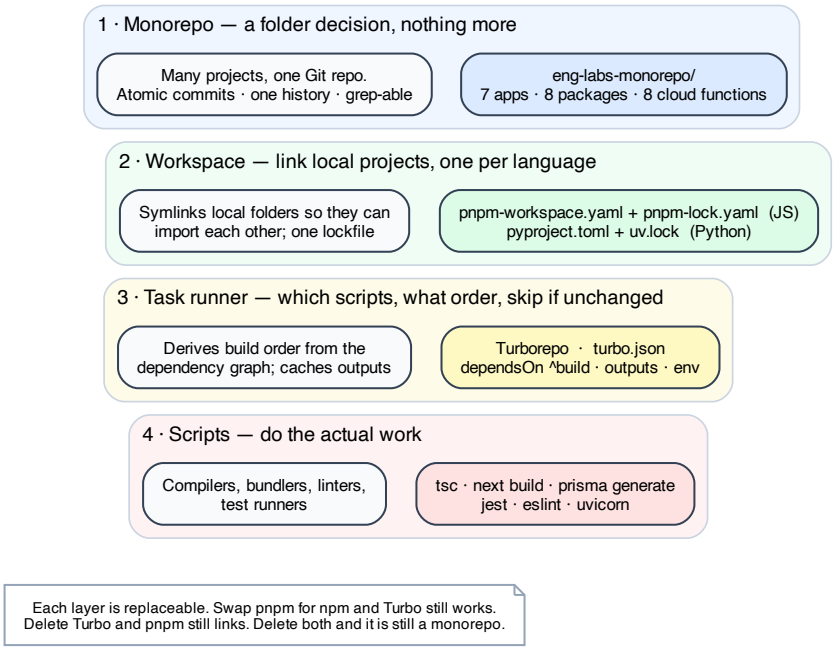

Figure 1: The four layers of a monorepo

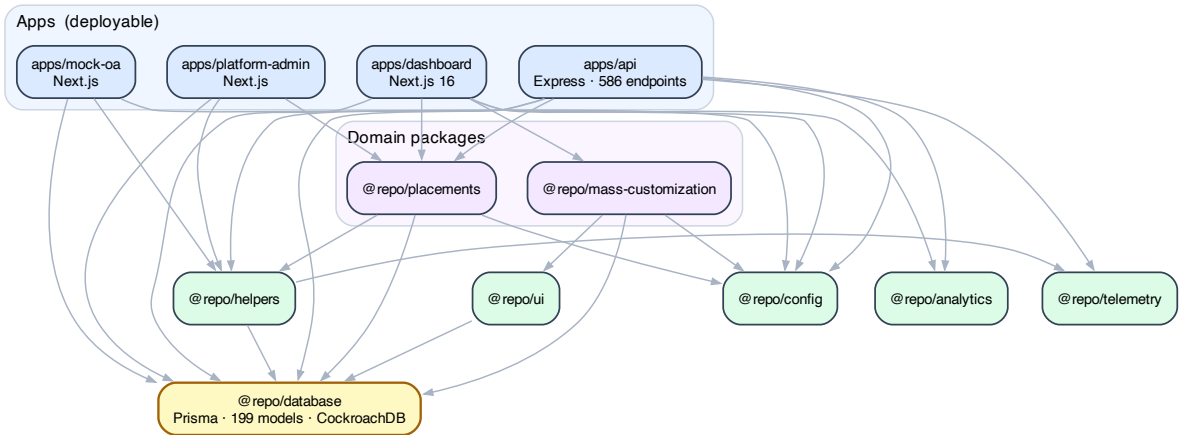

Figure 2: Workspace dependency graph

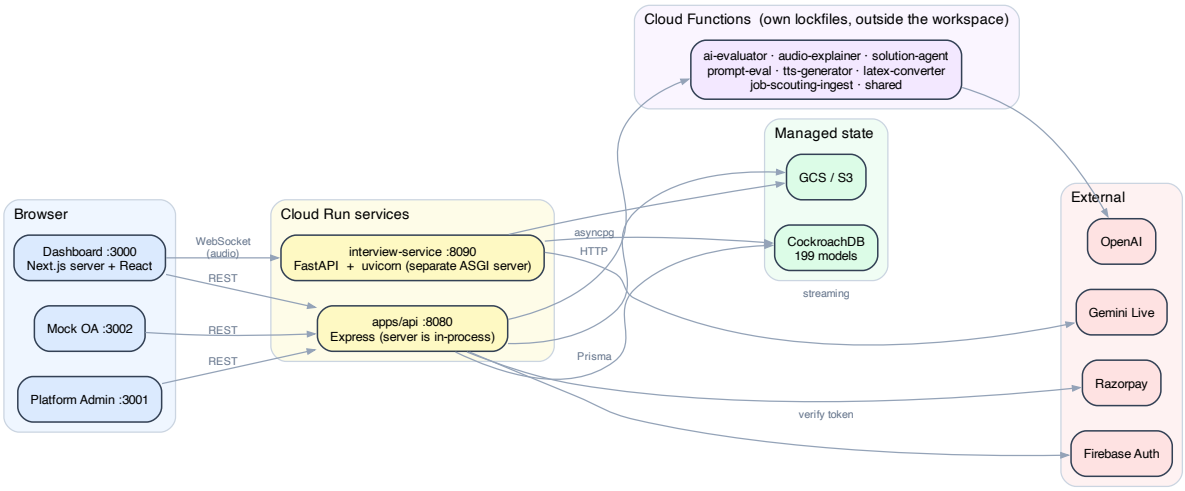

Figure 3: Deployed runtime topology

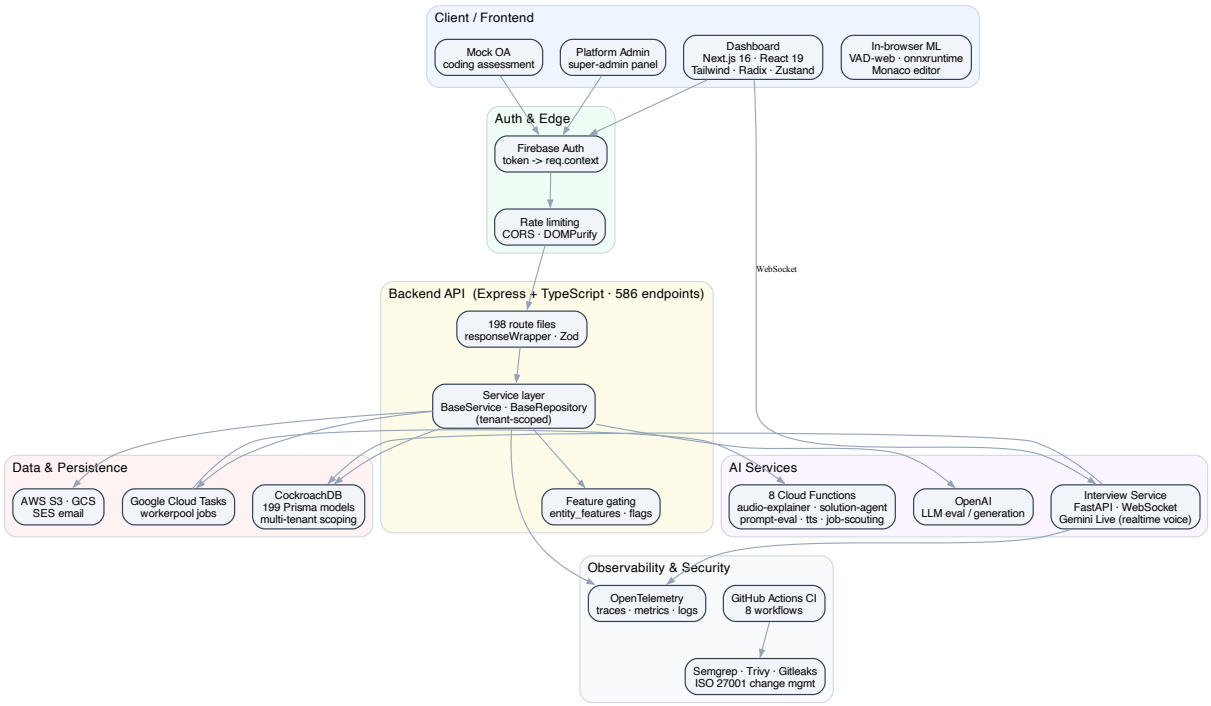

Figure 4: Eng Labs architecture & stack

4. The stack, by layer

- **Frontend** — Next.js 16 + React 19 dashboard (Tailwind, Radix UI, Zustand state, Monaco code editor, TipTap rich text, Recharts, Framer Motion). A standalone Next.js **Mock OA** coding-assessment app and a **Platform Admin** super-admin panel. Even runs **ML in the browser** — voice activity detection via `onnxruntime-web` + `vad-web` for the interview flow.
- **Backend API** — Express + TypeScript, **586 endpoints** across **198 route files**. Firebase token auth populating a request context (`userId`, `orgId`, `tenantId`, `role`, `features`), Zod validation, a uniform `responseWrapper` (792 call sites), rate limiting, input sanitization.
- **Service layer** — a phased refactor pulling routes into service classes (`ProjectsService`, `CoursesService`, `AdminStatsService`, `PersonalizationService`, …) on top of a `BaseRepository` that **auto-scopes every query by `organisation_id`**. Rolled out behind feature flags with legacy fallbacks kept verbatim for rollback safety.
- **Data** — **199 Prisma models** (6,444-line schema) on **CockroachDB** (distributed SQL), nanoid PKs, strict multi-tenant hierarchy (Organisation → Entity → Unit → Users).
- **AI services** — a **FastAPI + WebSocket interview service** running **Gemini Live** for real-time spoken mock interviews (prompt registry, stage manager, persona engine, answer scorer, session outbox); OpenAI for evaluation/generation; 8 cloud functions.
- **Code execution** — sandboxed compiler/runner services for C, C++, Java, JavaScript, Python and Verilog, plus a testcase evaluator and an executor dispatch client.
- **Integrations** — Razorpay payments, AWS S3 + GCS storage, SES email, 100ms video, Google Cloud Tasks job queue, PDF generation (pdfkit/puppeteer/LaTeX/KaTeX).
- **Observability** — a custom **OpenTelemetry** package (`@repo/telemetry`: traces, metrics, structured logs with PII redaction) + Sentry, enforced by CI review rules.
- **Security & delivery** — 8 GitHub Actions workflows, a `pre-push.sh` mirroring CI, a **90% coverage gate**, and a full security pipeline: **Semgrep** (SAST), **Trivy** (dependency CVEs), **Gitleaks** (secrets), pnpm supply-chain hardening, and a PR template for ISO 27001 change-management.

---

5. Multi-tenancy — the core architectural decision

```
Organisation → Entity (tenant) → Unit → Users
```

Every query scopes by `organisation_id`, and typically `tenant_id`.

The trap worth never forgetting: throughout the codebase "tenant_id" means `organisation_entities.id`. But `organisation_entities` *also has a column literally named* `tenant_id`, which is a legacy external identifier and must never be used for FKs or queries. Every `tenant_id` FK on every other table points at `organisation_entities.id`.

This is a textbook **unknown unknown** — nothing in the type system or ORM warns you. It is currently documented in `CLAUDE.md`; it belongs as a doc comment on the schema field itself.

Feature gating is per-tenant via the `entity_features` table, joined to a global `feature_registry`. Two gates must both pass: platform-level (`feature_registry.enabled`) and tenant-level (`entity_features.enabled`).

Authorization is layered middleware:

Guard	Call sites	Checks
`requireAuth()`	296	a valid `req.context` exists
`checkFeature(key)`	162	tenant + platform feature gates (DB query)
`requireFeature(key)`	47	feature flag from `req.context.features`
`requireRole(...roles)`	42	role membership
`requireModuleAdminAccess(module, path?)`	38	module admin + nested JSON permission
`requirePlacementPermission(...keys)`	4	granular permission keys

---

6. Request lifecycle

```
request
  → authMiddleware        populates req.context {userId, orgId, tenantId, unitId, role, features
  → requireAuth()         401 if no context
  → requireRole(...)      403 if wrong role
  → checkFeature(key)     403 if feature off for this tenant
  → handler
      → responseWrapper(req, res, async () => ({ responseData }))
          · maps Prisma P2025 → 404, P2002 → 409 (once, for all 792 sites)
          · Sentry capture, structured logging
          · writes an api_logs row per request
```

6.5 Domain modelling — Project Mentor as the worked example

The clearest module for explaining *domain* modelling (as opposed to tenancy or infrastructure), and the one with a design lesson worth repeating.

One container, three levels

```
Project (= Track, project_tracks)     ← Incharge OWNS this
      Team (project_groups)          ← Mentor GUIDES these
             Student (group_members) ← Learner WORKS here, in one team
```

There is no separate "project" vs "track" object — `project_tracks` is the single container, **labelled differently per audience** (learners and mentors see "Project"; the incharge sees "Track"). That relabel is UI-only; no table, column, model, route or store key was renamed.

A student is linked at **two** levels: `project_enrollments` (role `LEARNER` — the access gate) and `group_members` (the team they work in). Idea, milestones, submissions, chat and mentor all attach to the **team**.

Ownership and mentorship are orthogonal — deliberately

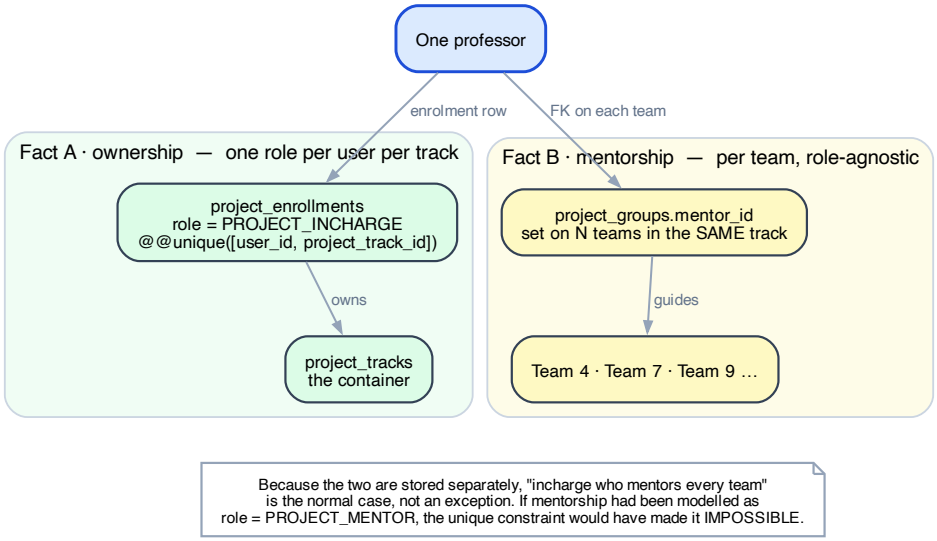

Figure 5: Ownership and mentorship are independent facts

Fact	Stored as	Cardinality
"I own this track"	`project_enrollments.role = PROJECT_INCHARGE`	**one role per user per track** — `@@unique([user_id, project_track_id])`
"I mentor this team"	`project_groups.mentor_id`	per team, independent

Because mentorship is an attribute of the **team** rather than a role on the **container**, "incharge who also mentors every team" is the normal case, not an exception. The code treats it that way already — `apps/api/src/routes/projects/admin-dashboard.ts:316` notes that a mentor is *"whoever is the team's `mentor_id` (role-agnostic), so an incharge acting as mentor is counted."*

The near-miss: had mentorship been modelled as `role = PROJECT_MENTOR` in enrolments, the unique constraint would have made that overlap **impossible to express**. This is the single best example in the codebase of a modelling choice that quietly preserved a capability.

> **Generalisable rule:** put a relationship on the entity it belongs to, not as a role on the container above it. Roles on a container tend to acquire uniqueness constraints; relationships on an entity don't.

Two open ambiguities worth closing

- **`mentor_id` is nullable**, so "no mentor assigned yet" and "mentored by the incharge" are indistinguishable. `admin-dashboard.ts:173` already special-cases `!g.mentor_id` as "no mentor." Decide explicitly whether `null` means unassigned or implies the incharge.
- **`mentor_id` is a single FK** — no co-mentors and no history. Reassigning a mentor silently erases the previous one, so "who reviewed this last semester" is currently unanswerable. That's a join table if it ever matters.

The surface architecture that follows from the model

Because the two roles overlap, the UI is split by **job**, not by role — four surfaces, any of which one person may use in a single session:

Surface	Job	Ordered by
Track level	configure & oversee the cohort (**non-evaluative**)	config, aggregate, exceptions
My Teams list	"who needs me right now?"	status, grouped by track
Review Queue	"give me the next thing to review"	urgency, across all tracks
Team workspace	work on one team	that team's timeline

One hard rule holds it together: **evaluation happens at team scope only — for everyone, including the incharge who mentors every team.** Full reasoning, plus the group-don't-nest and route-don't-filter decisions, in user-experience Part II.

---

7. Skills demonstrated

Grouped by category. **Importance** = how much the market values it. **AI shift** = how the skill changes as AI coding tools mature (this is the part worth internalizing).

Languages & Frameworks

Skill	Level	Why it matters	How it evolves with AI
TypeScript (end-to-end)	Proficient	The lingua franca of modern web; type-safety across a 477k-line codebase is what keeps it maintainable	AI writes the boilerplate; your value moves to *designing the types and contracts* it fills in. Strong typing is what lets AI edit safely.
Node.js / Express (API)	Proficient	Backbone of most web backends; 586 endpoints is serious surface area	AI scaffolds endpoints fast — the differentiator becomes API *design*, consistency, and knowing what not to build
React / Next.js 16	Proficient	Dominant frontend stack; App Router + Server Components is current	AI generates components well; judgment on state architecture, performance, and UX shifts up in value
Python (FastAPI, async)	Practiced→Proficient	The language of AI/ML services; async + WebSocket is non-trivial	Stays central — most AI tooling is Python-first

Concepts & Domains

Skill	Level	Why it matters	How it evolves with AI
Multi-tenant SaaS architecture	Proficient	Hard to get right, expensive to get wrong (data-leak = company-ending); a genuine senior signal	AI can't own tenant-isolation guarantees for you — this becomes *more* valuable as AI writes more code that must respect the boundary
Monorepo architecture (Turborepo/pnpm, polyglot)	Proficient	How serious teams manage many apps; being able to explain workspace vs task runner vs lockfile from first principles is rarer than using them	Stable infra skill; AI benefits from clear workspace boundaries
Data modeling (199 models)	Proficient	The schema outlives every framework; good modeling is a durable senior skill	AI suggests schemas, but modeling a real domain with integrity + migration safety stays human-led
Service-oriented refactoring	Proficient	Taming a monolith with feature flags + rollback paths is exactly what staff engineers do	AI accelerates the mechanical edits; the *strategy* (what to extract, in what order, how to de-risk) is the skill
Module design (interface vs implementation)	Practiced→Proficient	Deep modules, information hiding, pulling complexity downward — the difference between "works" and "stays cheap to change"	Rises sharply — AI writes more code, so the cost of a leaky abstraction multiplies faster
API design & contracts	Proficient	Everything integrates through APIs; consistency compounds	Rises in value — clean contracts are what make AI-generated clients/tests reliable
Realtime & AI integration (LLM, Gemini Live, WebSocket voice)	Proficient	The most in-demand skill category right now	This IS the AI-native skill — orchestrating models, streaming, evals, cost/latency tradeoffs
Payments / third-party integration	Practiced	Revenue-critical, correctness-critical	Stable; AI helps but you own the money-path correctness

Tools & Platforms

Skill	Level	Why it matters	How it evolves with AI
Prisma + CockroachDB	Proficient	ORM fluency + distributed SQL is a strong, transferable combo	AI writes queries; you own indexing, N+1 avoidance, migration safety

Skill	Level	Why it matters	How it evolves with AI
Turborepo / pnpm monorepo	Proficient	Ordered, cached builds across 15 workspaces; supply-chain hardening is a bonus signal	Stable
OpenTelemetry observability	Practiced→Proficient	Operational maturity — "can you debug it in prod?" separates senior from mid	AI can add instrumentation, but knowing *what* to measure is the skill
AWS/GCP (Cloud Run, Cloud Build, S3, SES, GCS, Cloud Tasks, Secret Manager, BigQuery)	Practiced	Cloud fluency is table-stakes for backend roles	Stable
Firebase Auth / JWT	Proficient	Auth is where security bugs live; getting it right is trust	Rises — auth correctness is exactly what you don't delegate blindly

Security, Quality & Delivery

Skill	Level	Why it matters	How it evolves with AI
Automated testing (Jest, 90% gate, "triangle coverage": happy/401/403)	Proficient	A hard coverage gate + auth-failure tests is a professional-grade quality culture	**Grows massively** — as AI writes more code, tests become the primary trust mechanism
DevSecOps (Semgrep/Trivy/Gitleaks, supply-chain, ISO 27001)	Practiced→Proficient	Security + compliance is a career moat; few devs actually do this	AI can triage findings, but owning the security posture is human
CI/CD (GitHub Actions, pre-push parity)	Proficient	Reliable delivery pipelines are what make teams fast	AI writes workflow YAML; pipeline *design* and gate philosophy is yours
AI-native / agentic development	Proficient	Building *with* agents (subagents, skills, review-rules, workflows) is the emerging meta-skill	This is the frontier — see agentic-development-handbook

---

8. Known gaps in *this document's* accuracy

Recorded because the failure mode is instructive and will recur.

- **Always confirm which branch you are measuring.** On 2026-08-11 a full review of this codebase was performed against a feature branch that was **181 commits and four months behind `main`**. It reported 149 models and 0 `requireRole` call sites; `main` had 199 models and 42. Nearly every "finding" was work that had already been done. **Run `git log --oneline -1 main` and `git rev-list --left-right --count main...HEAD` before drawing any conclusion from a codebase.**
- Figures here are a **snapshot**, not live. Re-measure before using them in an interview; the commands are in §1 of the lessons note.

---

9. Which roles these skills open

- **Founding / Product Engineer (startups)** — the strongest fit. Breadth across the whole stack plus shipping five products is exactly what early-stage companies pay a premium for.
- **AI / ML Application Engineer** — realtime LLM voice, evals, and AI services put you in the fastest-growing category. (This is *AI application* engineering, not model research — a distinction worth being honest about in interviews.)
- **Full-Stack Engineer** — the direct fit; both sides shipped end-to-end at real scale.
- **Backend / Platform Engineer** — 586 endpoints, 199-model schema, service architecture, and observability are a strong backend/platform story.
- **Senior / Staff Engineer / Tech Lead** — the multi-tenant architecture, the phased refactor with rollback safety, and the quality/security gates are senior-signal work. This is the level to aim for.
- **DevSecOps / Platform Reliability** — the security pipeline + telemetry + CI is a credible secondary track. Note the honest limit: no IaC (Terraform/K8s), so this is "uses cloud infra well," not "builds infra."

10. How these roles & skills evolve with AI — the honest read

1. **The floor rises, so lead with judgment, not typing.** AI writes competent code from a good spec. The scarce skills are the ones AI *can't* own: system design, multi-tenant safety guarantees, choosing what to build, API/data contracts, and knowing when the AI is wrong. Present the *decisions*, not the line count.

2. **Tests and types become the trust layer.** When AI produces most of the code, the code that verifies it (tests, types, schemas, CI gates) is where humans add the most leverage. The 90% gate + triangle-coverage culture is directly this skill — appreciating, not depreciating.

3. **AI-application engineering is its own discipline now.** Orchestrating LLMs, streaming realtime voice, writing evals, managing cost/latency/quality tradeoffs — the interview service is real evidence of a skill category that barely existed two years ago.

4. **Being "AI-native" is itself a differentiator.** This monorepo has custom subagents, skills, review-rules, and workflows. The next tier of engineers are the ones who can direct fleets of agents and still own correctness.

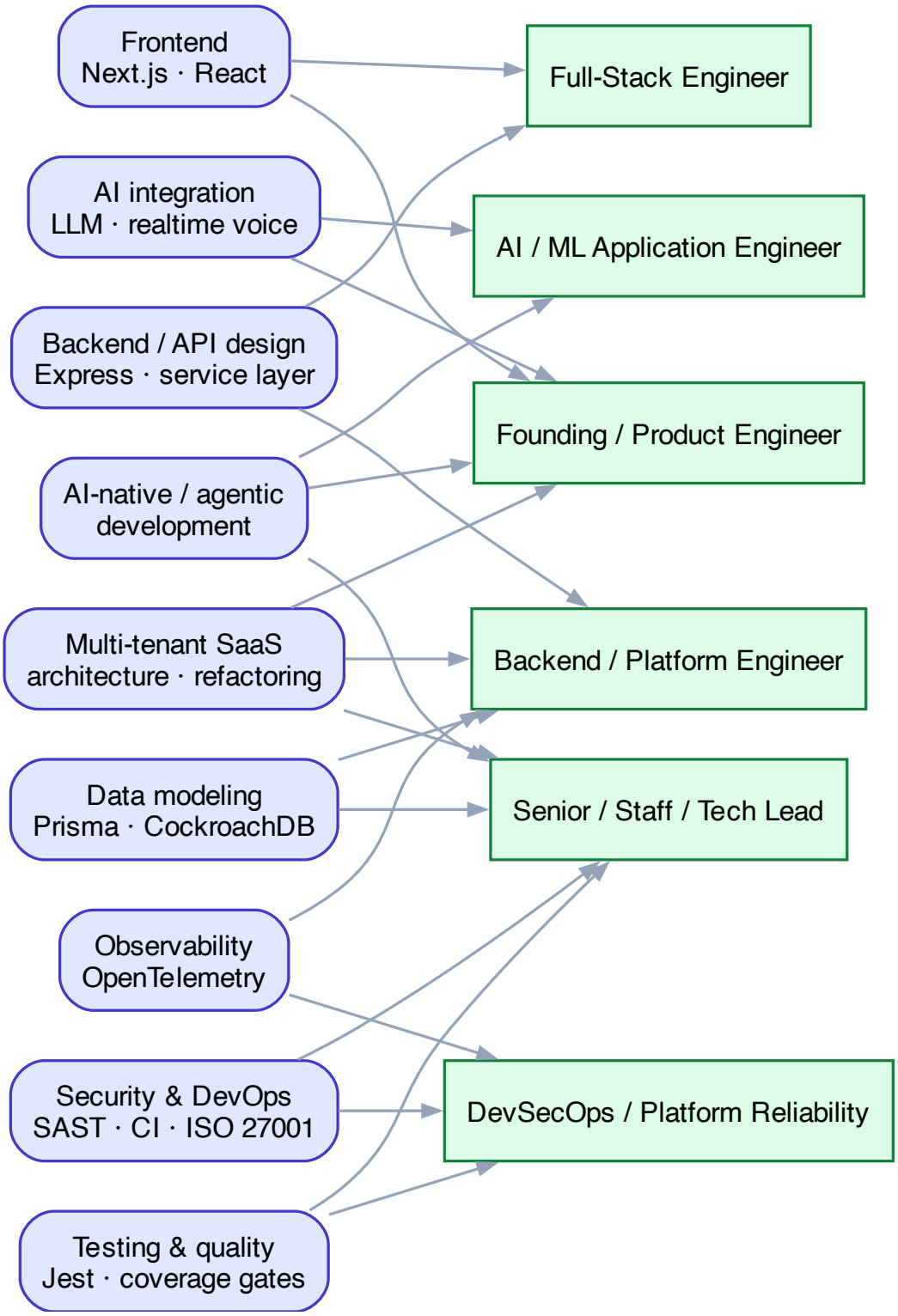

Figure 6: Skills → roles mapping

5. **Where to grow next** (Proficient → Expert, toward Staff): deeper distributed-systems fundamentals, owning a system's scalability/reliability story end-to-end, formal leadership evidence, and finishing the migrations that are still half-done (see the lessons note, §2.7).

Outcome / impact

A single codebase serving five products to multiple institutions, with tenant isolation, per-customer feature gating, a 90% test-coverage gate, full observability, and a security + change-management pipeline mature enough for ISO 27001. Delivered breadth (full-stack + AI + infra) usually spread across a whole team.

Skills demonstrated (summary for the matrix)

- TypeScript / Node / Express — Proficient
- React / Next.js — Proficient
- Python / FastAPI (async, WebSocket) — Practiced→Proficient
- Multi-tenant SaaS architecture — Proficient
- Monorepo architecture (Turborepo / pnpm, polyglot) — Proficient
- Data modeling (Prisma / CockroachDB) — Proficient
- Service-oriented refactoring — Proficient
- Module design / information hiding — Practiced→Proficient
- AI / LLM application engineering (OpenAI, Gemini Live realtime) — Proficient
- Automated testing & quality gates — Proficient
- DevSecOps (SAST/DAST, supply-chain, ISO 27001) — Practiced→Proficient
- Observability (OpenTelemetry) — Practiced→Proficient
- CI/CD (GitHub Actions, monorepo) — Proficient
- AI-native / agentic development — Proficient

Artifacts

- Codebase: `eng-labs-monorepo` (private) — `apps/{api,dashboard,interview-service,mock-oa,platform-packages/*`, `cloud-functions/*`
- Module docs: `documentation_personal/` (mass-customization, digital-evaluation-system, project-mentor, placements, platform-core, exam-proctoring, api-development-quality-report)
- Companion note: multi-tenancy
- Related note: agentic-development-handbook

Last updated: 2026-08-11